

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 12 – SEARCHING



NAMA : Muhammad Ervan Febriano

NIM : 109082500079

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Searching atau pencarian adalah proses menemukan data tertentu dari sekumpulan data yang tersimpan di dalam suatu struktur data, seperti array. Dalam pemrograman, proses pencarian digunakan untuk mengetahui apakah suatu data ada, menemukan posisi data, ataupun mengambil informasi tertentu berdasarkan kriteria yang dicari. Pada proses searching, terdapat kemungkinan data yang dicari ditemukan maupun tidak ditemukan. Modul ini membahas dua metode pencarian utama, yaitu *Sequential Search* dan *Binary Search*.

- Sequential Search

Sequential Search atau pencarian sekuensial adalah metode pencarian yang dilakukan dengan mengecek data satu per satu mulai dari elemen pertama hingga elemen terakhir, atau sampai data yang dicari ditemukan. Metode ini tidak memerlukan data dalam keadaan terurut sehingga lebih fleksibel digunakan. Namun, jika jumlah data sangat banyak, proses pencarian bisa menjadi lebih lambat karena harus membandingkan elemen satu per satu.

- Binary Search

Binary Search adalah metode pencarian yang bekerja dengan membagi data menjadi dua bagian secara terus-menerus hingga data yang dicari ditemukan. Algoritma ini hanya dapat digunakan pada data yang sudah terurut, baik secara ascending maupun descending. Binary Search lebih efisien dibandingkan Sequential Search karena jumlah perbandingan yang dilakukan lebih sedikit, terutama pada data berukuran besar.

- Searching pada Tipe Data Struct

Selain pada tipe data dasar, algoritma searching juga dapat diterapkan pada array yang berisi tipe data struct. Pada proses ini, pencarian dilakukan berdasarkan field tertentu, seperti nama atau NIM mahasiswa. Khusus Binary Search, field yang digunakan untuk mencari harus sesuai dengan urutan data pada array agar algoritma dapat berjalan dengan benar.

B. Guided

1. [guided1.go]

```
package main

import "fmt"

type arrData [5]string //tipe data alias
```

```

func seqSearch(arr arrData, binatangCari string) int {
    var found int = -1
    for i := 0; i < len(arr); i++ {
        if arr[i] == binatangCari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var arrBinatang arrData //array tipe string

    for i := 0; i < len(arrBinatang); i++ {
        fmt.Printf("Masukkan data binatang indeks ke-%d : ", i)
        fmt.Scan(&arrBinatang[i])
    }
    fmt.Println()

    var binatangCari string
    fmt.Print("Masukkan nama binatang yang mau dicari : ")
    fmt.Scan(&binatangCari)

    var idxCari int
    idxCari = seqSearch(arrBinatang, binatangCari)

    if idxCari > -1 {
        fmt.Printf("Data %s ditemukan pada indeks ke-%d!",
binatangCari, idxCari)
    } else if idxCari == -1 {
        fmt.Printf("Data %s tidak ditemukan!", binatangCari)
    }
}

```

Output :

```
PS C:\Users\USER\Documents\ENGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febria
\Documents\ENGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\109082500079_
Masukkan data binatang indeks ke-0 : Kambing
Masukkan data binatang indeks ke-1 : Domba
Masukkan data binatang indeks ke-2 : Kucing
Masukkan data binatang indeks ke-3 : Anjing
Masukkan data binatang indeks ke-4 : none

Masukkan nama binatang yang mau dicari : Anjing
Data Anjing ditemukan pada indeks ke-3!
```

2. [guided2.go]

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim   string
    IPK   float64
}

type arrMahasiswa [3]mahasiswa

func IPKTerbesar(array []mahasiswa) (mahasiswa, int) {
    var mhsTerbesar mahasiswa = array[0]
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK > mhsTerbesar.IPK {
            mhsTerbesar = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerbesar, idxDitemukan
}

func IPKTerkecil(array []mahasiswa) (mahasiswa, int) {
    var mhsTerkecil mahasiswa = array[0]
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK < mhsTerkecil.IPK {
```

```

        mhsTerkecil = array[i]
        idxDitemukan = i
    }
}
return mhsTerkecil, idxDitemukan
}

func main() {
    var arrMhs arrMahasiswa
    var i, j int
    for i = 0; i < len(arrMhs); i++ {
        fmt.Println("DATA MAHASISWA INDEKS KE-", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&arrMhs[i].nama)
        fmt.Print("Masukkan NIM : ")
        fmt.Scan(&arrMhs[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&arrMhs[i].IPK)
        fmt.Println("=====")
    }

    fmt.Println()
    for j = 0; j < len(arrMhs); j++ {
        fmt.Println("DATA MAHASISWA INDEKS KE-", j)
        fmt.Println("Nama : ", arrMhs[j].nama)
        fmt.Println("NIM : ", arrMhs[j].nim)
        fmt.Println("IPK : ", arrMhs[j].IPK)
        fmt.Println("=====")
    }
    fmt.Println()

    var mhsMaks, mhsMin mahasiswa
    var idxMaks, idxMin int

    mhsMaks, idxMaks = IPKTerbesar(arrMhs[:])
    mhsMin, idxMin = IPKTerkecil(arrMhs[:])

    fmt.Println("=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===")
    fmt.Printf("IPK Terbesar : %.2f, atas nama %s dengan NIM %s,
ditemukan pada indeks ke-%d", mhsMaks.IPK, mhsMaks.nama, mhsMaks.nim,
idxMaks)
    fmt.Printf("IPK Terkecil : %.2f, atas nama %s dengan NIM %s,
ditemukan pada indeks ke-%d", mhsMin.IPK, mhsMin.nama, mhsMin.nim,
idxMin)
}

```

```

● PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\109082500079_Muhammad_Ervan_Febriano\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\109082500079_Muhammad_Ervan_Febriano>
Jumlah data: 3
Masukkan data terurut:
1
2
3
Angka yang dicari: 2
Data ditemukan di index: 1

```

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim int
    IPK float64
}

type arrMahasiswa [5]mahasiswa

func binSearchStruct(arrData arrMahasiswa, nimDicari int) int {
    var found int = -1
    var med int
    var kr int = 0
    var kn int = len(arrData) - 1
    for kr <= kn && found == -1 {
        med = (kr + kn) / 2
        if nimDicari < arrData[med].nim {
            kn = med - 1
        } else if nimDicari > arrData[med].nim {
            kr = med + 1
        } else {
            found = med
        }
    }
    return found
}
```

```

func seqSearchStruct(arrData arrMahasiswa, nimDicari int) int {
    var found int = -1
    for i := 0; i < len(arrData); i++ {
        if arrData[i].nim == nimDicari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var mahasiswa06 arrMahasiswa
    var nimDicari int

    fmt.Println("--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---")
    for i := 0; i < len(mahasiswa06); i++ {
        fmt.Printf("=== Masukkan data mahasiswa 06 pada indeks ke-%d\n", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&mahasiswa06[i].nama)
        fmt.Print("Masukkan NIM : ")
        fmt.Scan(&mahasiswa06[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&mahasiswa06[i].IPK)
        fmt.Println("-----")
    }
    fmt.Println()

    fmt.Print("Masukkan NIM mahasiswa yang ingin dicari : ")
    fmt.Scan(&nimDicari)
    fmt.Println()

    var idxHasilCariSeqSearch int
    idxHasilCariSeqSearch = seqSearchStruct(mahasiswa06, nimDicari)

    fmt.Println("== HASIL SEQUENTIAL SEARCH ==")
    if idxHasilCariSeqSearch > -1 {
        fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di indeks ke-%d!\n", nimDicari, idxHasilCariSeqSearch)
        fmt.Println("Berikut Detail Datanya : ")
        fmt.Printf("Nama : %s\n", mahasiswa06[idxHasilCariSeqSearch].nama)
    }
}

```

```

        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariSeqSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariSeqSearch].IPK)
    } else if idxHasilCariSeqSearch == -1 {
        fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada array!",
nimDicari)
    }
    fmt.Println()

    var idxHasilCariBinSearch int
    idxHasilCariBinSearch = binSearchStruct(mahasiswa06, nimDicari)

    fmt.Println("== HASIL BINARY SEARCH ==")
    if idxHasilCariBinSearch > -1 {
        fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di
indeks ke-%d!\n", nimDicari, idxHasilCariBinSearch)
        fmt.Println("Berikut Detail Datanya : ")
        fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariBinSearch].nama)
        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariBinSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariBinSearch].IPK)
    } else if idxHasilCariBinSearch == -1 {
        fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada array!",
nimDicari)
    }
}

```


Output:

```
PS C:\Users\USER\Documents\ENGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\109082500079_Muhammad_Ervan_Febriano\week 11\guided\guided 3\
--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---
=== Masukkan data mahasiswa 06 pada indeks ke-0 ===
Masukkan nama : andi
Masukkan NIM : 101
Masukkan IPK : 3
-----
=== Masukkan data mahasiswa 06 pada indeks ke-1 ===
Masukkan nama : ervan
Masukkan NIM : 102
Masukkan IPK : 3
-----
=== Masukkan data mahasiswa 06 pada indeks ke-2 ===
Masukkan nama : yazid
Masukkan NIM : 103
Masukkan IPK : 3
-----
=== Masukkan data mahasiswa 06 pada indeks ke-3 ===
Masukkan nama : hanif
Masukkan NIM : 104
Masukkan IPK : 2.5
-----
=== Masukkan data mahasiswa 06 pada indeks ke-4 ===
Masukkan nama : faiq
Masukkan NIM : 105
Masukkan IPK : 2.8
-----

Masukkan NIM mahasiswa yang ingin dicari : 102

== HASIL SEQUENTIAL SEARCH ==
Data NIM mahasiswa 102 ditemukan pada array di indeks ke-1!
Berikut Detail Datanya :
Nama : ervan
NIM : 102
IPK : 3.00

== HASIL BINARY SEARCH ==
Data NIM mahasiswa 102 ditemukan pada array di indeks ke-1!
Berikut Detail Datanya :
Nama : ervan
NIM : 102
IPK : 3.00
```

C. Unguided

1. Soal Unguided 1

Pada pemilihan ketua RT yang baru saja berlangsung, terdapat 20 calon ketua yang bertanding memperebutkan suara warga. Perhitungan suara dapat segera dilakukan karena warga cukup mengisi formulir dengan nomor dari calon ketua RT yang dipilihnya. Seperti biasa, selalu ada pengisian yang tidak tepat atau dengan nomor pilihan di luar yang tersedia, sehingga data juga harus divalidasi. Tugas Anda untuk membuat program mencari siapa yang memenangkan pemilihan ketua RT.

Buatlah program pilkart yang akan membaca, memvalidasi, dan menghitung suara yang diberikan dalam pemilihan ketua RT tersebut.

Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah integer dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian sejumlah baris yang mencetak data para calon apa

Jawaban :

```
package main

import "fmt"

const NMAX int = 21

func main() {
    var suara [NMAX]int
    var x int
    var totalMasuk, totalSah, i int

    fmt.Scan(&x)

    for x != 0 {
        totalMasuk++

        if x >= 1 && x <= 20 {
            suara[x]++
            totalSah++
        }

        fmt.Scan(&x)
    }

    fmt.Println("Suara masuk:", totalMasuk)
    fmt.Println("Suara sah:", totalSah)

    for i = 1; i <= 20; i++ {
        if suara[i] > 0 {
            fmt.Println(i, ":", suara[i])
        }
    }
}
```

Output :

```
PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febrian  
IKUM\109082500079_Muhammad_Ervan_Febriano\109082500079_Muhammad_Ervan_Febriano\week 11\  
7 19 3 2 78 3 1 -3 18 19 0  
Suara masuk: 10  
Suara sah: 8  
1 : 1  
2 : 1  
3 : 2  
7 : 1  
18 : 1  
19 : 2
```

Penjelasan :

[Program di atas adalah aplikasi penghitung suara sederhana untuk pemilihan ketua RT yang ditulis menggunakan bahasa Go. Cara kerjanya cukup simpel: program akan terus membaca angka yang dimasukkan satu per satu sampai mendeteksi angka 0 yang berfungsi sebagai penanda berhenti atau setop. Setiap angka yang masuk akan dihitung ke dalam "total suara masuk", tetapi hanya angka dari rentang 1 sampai 20 saja yang dianggap sebagai "suara sah" dan poinnya dimasukkan ke dalam keranjang perolehan masing-masing kandidat (*array*). Begitu proses input selesai, program bakal langsung menampilkan rekapitulasi akhirnya, mulai dari keseluruhan jumlah suara yang masuk, jumlah suara yang sah, sampai rincian perolehan suara buat tiap calon yang berhasil mendapatkan minimal satu dukungan.]

2. Soal Unguided 2

Maaf, saya baru sadar di teks aslinya potongan kode dan nomor halaman PDF nyelip persis di tengah-tengah kalimat. Ini susunan paragraf yang sudah dirapikan dan disambung dengan benar:

Berdasarkan program sebelumnya, buat program pilkart yang mencari siapa pemenang pemilihan ketua RT. Sekaligus juga ditentukan bahwa wakil ketua RT adalah calon yang mendapatkan suara terbanyak kedua. Jika beberapa calon mendapatkan suara terbanyak yang sama, ketua terpilih adalah dengan nomor peserta yang paling kecil dan wakilnya dengan nomor peserta terkecil berikutnya.

Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah bilangan bulat dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian tercetak calon nomor berapa saja yang menjadi pasangan ketua RT dan wakil ketua RT yang baru.

Jawaban :

```
package main
import "fmt"

const NMAX int = 21

func main() {
    var suara [NMAX]int
    var x, totalMasuk, totalSah int
    var i, ketua, wakil int

    fmt.Scan(&x)

    for x != 0 {
        totalMasuk++

        if x >= 1 && x <= 20 {
            suara[x]++
            totalSah++
        }

        fmt.Scan(&x)
    }

    ketua = 1
    for i = 2; i <= 20; i++ {
        if suara[i] > suara[ketua] {
            ketua = i
        }
    }

    wakil = -1
    for i = 1; i <= 20; i++ {
        if i != ketua {
            if wakil == -1 || suara[i] > suara[wakil] {
                wakil = i
            }
        }
    }
}
```

```

    fmt.Println("Suara masuk:", totalMasuk)
    fmt.Println("Suara sah:", totalSah)
    fmt.Println("Ketua RT:", ketua)
    fmt.Println("Wakil ketua:", wakil)
}

```

Output :

```

PS C:\Users\USER\Documents\ENGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\109082500079_Muhammad_Ervan_Febriano\week 11\unguided 3>
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19

```

Penjelasan :

[Program ini berfungsi untuk menghitung hasil pemilihan RT sekaligus menentukan posisi ketua dan wakilnya secara otomatis. Prosesnya dimulai dengan menerima input angka terus-menerus sampai muncul angka 0 sebagai penanda berhenti, di mana hanya angka 1 sampai 20 saja yang dihitung sebagai suara sah ke masing-masing kandidat. Setelah semua suara terkumpul, program menelusuri data untuk mencari peraih suara terbanyak pertama sebagai ketua (jika ada yang seri, kandidat bernomor urut lebih kecil akan diutamakan). Selanjutnya, program melakukan pencarian serupa untuk mencari peraih suara terbanyak kedua sebagai wakil ketua dengan cara mengecualikan kandidat yang sudah terpilih jadi ketua, lalu mencetak semua rekapitulasi akhirnya.]

3. Soal Unguided 3

Diberikan n data integer positif dalam keadaan terurut membesar dan sebuah integer lain k , apakah bilangan k tersebut ada dalam daftar bilangan yang diberikan? Jika ya, berikan indeksnya, jika tidak sebutkan "TIDAK ADA".

Masukan terdiri dari dua baris. Baris pertama berisi dua buah integer positif, yaitu n dan k . n menyatakan banyaknya data, dimana $1 < n \leq 1000000$. k adalah bilangan yang ingin dicari. Baris kedua berisi n buah data integer positif yang sudah terurut membesar.

Keluaran terdiri dari satu baris saja, yaitu sebuah bilangan yang menyatakan posisi data yang dicari (k) dalam kumpulan data yang diberikan. Posisi data dihitung dimulai dari angka 0. Atau memberikan keluaran "TIDAK ADA" jika data k tersebut tidak ditemukan dalam kumpulan.

Jawaban:

```

package main

import "fmt"

const NMAX = 1000000

var data [NMAX]int

func main() {
    var n, k int

    fmt.Scan(&n, &k)

    isiArray(n)

    pos := posisi(n, k)

    if pos == -1 {
        fmt.Println("TIDAK ADA")
    } else {
        fmt.Println(pos)
    }
}

func isiArray(n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

func posisi(n, k int) int {
    kiri := 0
    kanan := n - 1

    for kiri <= kanan {
        tengah := (kiri + kanan) / 2

        if data[tengah] == k {
            return tengah
        } else if data[tengah] < k {
            kiri = tengah + 1
        } else {
            kanan = tengah - 1
        }
    }
}

```

```
    return -1  
}
```

Output :

```
● PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\109082500079_Muhammad_Ervan_Febriano  
12 534  
1 3 8 16 32 123 323 323 534 543 823 999  
8  
● PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\109082500079_Muhammad_Ervan_Febriano  
12 535  
1 3 8 16 32 123 323 323 534 543 823 999  
TIDAK ADA
```

Penjelasan :

[Program ini merupakan implementasi dari algoritma pencarian bagi dua (*binary search*) untuk menemukan posisi suatu angka spesifik di dalam kumpulan data yang sudah terurut. Prosesnya diawali dengan membaca jumlah data (n) dan angka target yang ingin dicari (k), yang kemudian dilanjutkan oleh fungsi isiArray untuk menerima masukan deretan angkanya satu per satu ke dalam *array*. Setelah data siap, fungsi posisi akan bertugas melakukan pencarian dengan cara menebak nilai tengahnya secara berulang; membuang setengah porsi data (kiri atau kanan) yang sudah pasti salah hingga angkanya berhasil dikunci. Jika angka tersebut ditemukan, program akan mencetak posisi indeks letak angka itu berada, namun jika rentang pencarian sudah habis dan angkanya terbukti tidak ada, program akan langsung mencetak teks "TIDAK ADA".]

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penyelesaian kasus pencarian data spesifik harus selalu mempertimbangkan kemungkinan tidak ditemukannya data di dalam kumpulan tersebut. Pemilihan jenis algoritma sangat bergantung pada kondisi awal data yang ada. Metode *Sequential Search* sangat fleksibel untuk diterapkan pada data yang acak, namun pengecekan satu per satu membuatnya memakan waktu lebih lama untuk data berskala besar. Di sisi lain, metode *Binary Search* mampu menemukan data jauh lebih efisien dengan membuang setengah rentang pencarian setiap kali menebak, namun memiliki kelemahan karena mewajibkan datanya sudah terurut secara spesifik, terutama jika berurusan dengan tipe data *struct*. Secara keseluruhan, implementasi algoritma pencarian yang tepat harus mampu memberikan indikator status penemuan yang

jelas, baik dengan memunculkan posisi indeks data yang akurat maupun memberikan nilai *default* (seperti -1) ketika data tersebut memang tidak tersedia.